

SIGN LANGUAGE TRANSLATION SYSTEM

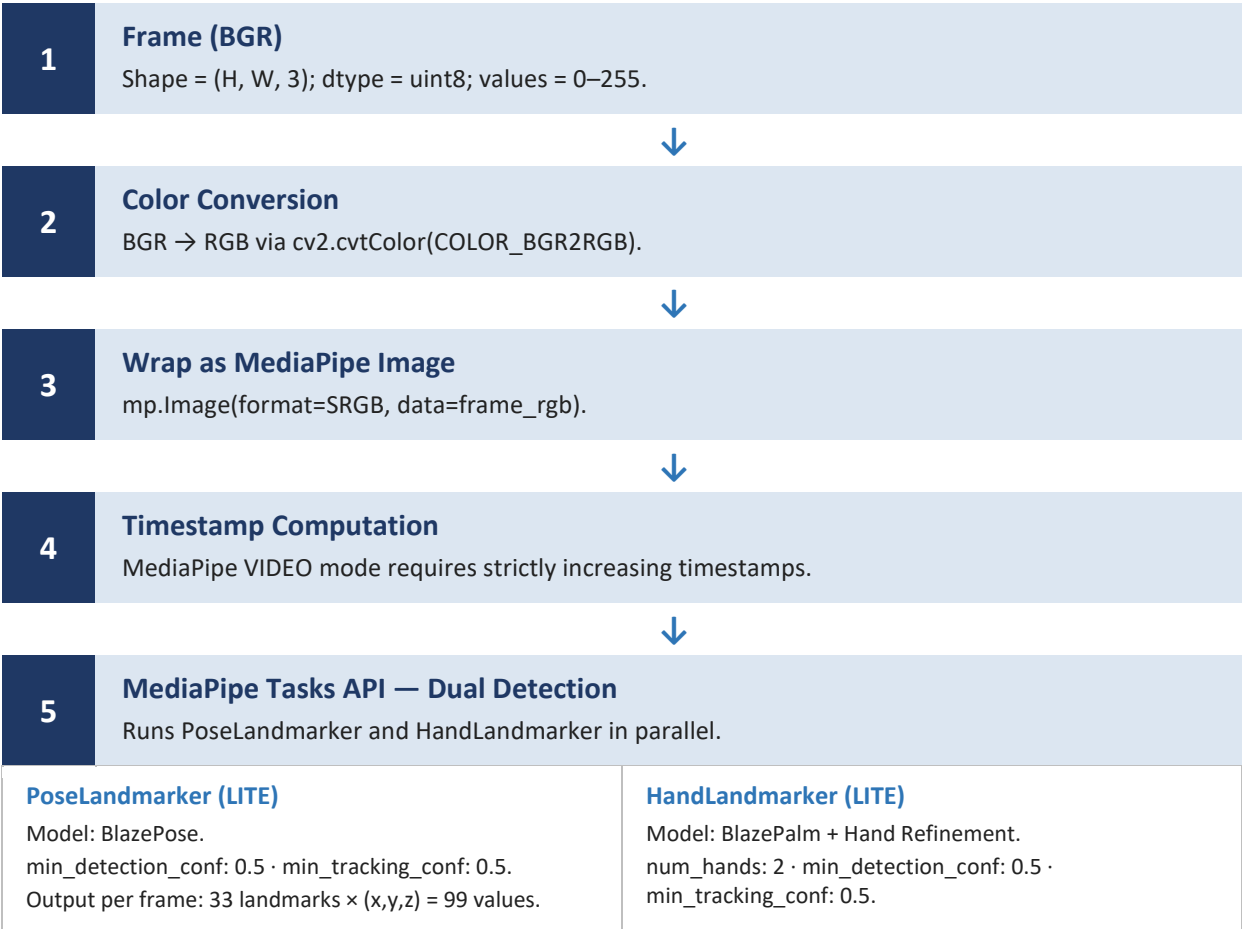
SLR Component

Complete Architecture Flow Diagram

Pipeline Stages (Reference)

#	Stage
1	Frame Prep
2	Landmark Extraction
3	Landmark Preparation
4	Resampling
5	Transformer Encoder
6	Cleaning Feature Vector & Pooling
7	Feed Forward Neural Network

1. Frame Preparation & Feature Extraction



Key indices: 11 = left_shoulder, 12 = right_shoulder.

Output per frame: up to 2 hands × 21 landmarks × (x,y,z) = up to 126 values.

Plus handedness label: "Left" or "Right".



6

Concatenate Raw Landmark Vectors

pose(99) + left_hand(63) + right_hand(63) + left_present(1) + right_present(1) = 227-dim vector.

Layout — [0:98] 33 pose × (x,y,z) = 99 values · [99:161] 21 left-hand × (x,y,z) = 63 values · [162:224]

21 right-hand × (x,y,z) = 63 values · [225] left_present flag · [226] right_present flag.



7

Normalize & Scale (relative to shoulders)

Step 1: locate shoulders at pose[11], pose[12].

Step 2: if both shoulders are (0,0,0) → return raw concatenation (skip normalization).

Step 3: center = (left_shoulder + right_shoulder) / 2; scale = ||left_shoulder – right_shoulder||, clamped to max(scale, 1e-6) to prevent divide-by-zero.

Step 4: all_landmarks = (all_landmarks – center) / scale.

Effect: translation + scale invariance; unit = "shoulder-widths" (person-independent).

Output: normalized_xyz → (75, 3) float32.



8

Flatten & Concatenate with Presence Flags

features = concat(normalized_xyz.flatten() [75×3 = 225 floats], [left_present, right_present] [2 binary flags]).astype(float32).

Shape: (227,) float32. hand_present = bool(left or right present).



9

Repeat for All T Frames in the Clip

Each frame → (227,); stacked across T frames → (T, 227) float32, where T = original clip frame count.

Proceeds to the Resampling Layer.

2. Resampling Layer

1

Input

(T, 227) float32, where T = original clip frame count (varies: typically 2 to 200+).



2

resample_sequence(features, target_len=64)

src_idx = linspace(0, T-1, T) — source positions. tgt_idx = linspace(0, T-1, 64) — target positions (same range).

For each of the 227 dimensions: np.interp(tgt_idx, src_idx, features[:, d]).

Method: per-dimension linear interpolation, preserving temporal continuity of motion trajectories.

Mean T before resampling ≈ 90 frames (WLASL100 clips average ~90 frames @ 30fps ≈ 3 seconds).



Case 1: $T > 64$ — Compress Longer clip is squeezed to 64 frames. Drops some intermediate information.	Case 2: $T < 64$ — Stretch Shorter clip is stretched to 64 frames via interpolation. Creates new intermediate frames.
Case 3: $T == 64$ — Skip Resampling Returns a copy of the original sequence. Also applies when $T == 1 \rightarrow$ repeat 64 times.	



3

Output

(64, 227), dtype = float32 — always exactly 64 frames after resampling.

Proceeds to the Transformer Encoder.

3. Transformer Encoder

1

Input Shape

(1, 64, 227) — batch = 1, 64 frames, 227 dims.



2

Linear Projection
227 $\rightarrow$ 192 via Linear(227, 192) weight matrix + bias.
Shape: (1, 64, 192).


3

Layer Normalization + Dropout
LayerNorm(192) + Dropout(p=0.2).
Shape: (1, 64, 192) — unchanged.


4

Positional Encoding
Sinusoidal PE (max_len = 128): $pe[:, 0::2] = \sin(pos / 10000^{(2i/d)})$; $pe[:, 1::2] = \cos(pos / 10000^{(2i/d)})$.
 $h = h + pe[:, :64, :]$. Shape: (1, 64, 192) — unchanged.


5

Multi-Head Self-Attention — 4 Transformer Encoder Layers

Each of the 4 layers (independent weights) repeats the same internal structure below.

Multi-Head Attention (4 heads per layer)

Head splitInput: (1, 64, 192), split into 4 heads $\times$ 48 dims each (Q, K, V per head).
For each head: $Q = h \cdot W_Q \rightarrow (1, 64, 48)$; $K = h \cdot W_K \rightarrow (1, 64, 48)$; scores = $Q \cdot K^T / \sqrt{48} \rightarrow (1, 64, 64)$; weights = $\text{softmax}(\text{scores}) \rightarrow (1, 64, 64)$; $V = h \cdot W_V \rightarrow (1, 64, 48)$; head_out = weights $\cdot V \rightarrow (1, 64, 48)$.

Total: 4 layers $\times$ 4 heads = 16 attention matrices, each (1, 64, 64) — a "frame-to-frame" map.
Heads are concatenated back to shape (1, 64, 192).

Add & LayerNorm (residual)

$h = \text{LayerNorm}(h + \text{attention_output})$. Shape: (1, 64, 192) — unchanged.

Feed-Forward Network (per layer)

$\text{Linear}(192 \rightarrow 512)$ — expand $\cdot$ GELU activation (smooth non-linearity) $\cdot$ Dropout($p=0.2$) $\cdot$ $\text{Linear}(512 \rightarrow 192)$ — project back.

Shape: (1, 64, 192) — unchanged.

Add & LayerNorm (residual)

$h = \text{LayerNorm}(h + \text{ff_output})$. Shape: (1, 64, 192) — unchanged.

Repeated identically for layers 2, 3, and 4. After 4 layers, every frame carries context from all 64 frames — the model has "watched" the entire sign sequence. Output: (1, 64, 192).



Proceeds to Cleaning & Pooling.

4. Cleaning Feature Vector & Pooling

1

Input

(1, 64, 192) — 64 context-aware frame vectors.



Left Branch: Mean Pooling (not weighted)

$h.\text{mean}(\text{dim}=1)$ — each of the 64 frames contributes equally.

Simple average over the time dimension.

Output: (1, 192).

Captures an "average" representation of the entire sign; all frames get weight $1/64$.

Right Branch: Attention Pooling (weighted)

$q = \text{learned_query param} \rightarrow (1, 1, 192)$, init $N(0, 0.02)$.

$\text{scores} = q \cdot h^T / \sqrt{192} \rightarrow (1, 1, 64)$; $\text{weights} =$

$\text{softmax}(\text{scores}) \rightarrow (1, 1, 64)$.

$\text{attn_pooled} = \text{weights} \cdot h \rightarrow (1, 1, 192)$, then squeeze $\rightarrow (1, 192)$.

Captures "peak" frames — the model learns which frames matter most (e.g. peak hand configuration).



2

Concatenate Both Pooled Vectors

$\text{combined} = \text{cat}([\text{attn}, \text{mean}], \text{dim}=-1)$. Shape: (1, 384) = $[\text{attn_pooled}(192), \text{mean_pooled}(192)]$.

Rationale: attention finds the "highlight reel"; mean finds the "average"; both together outperform either alone.

Proceeds to the Feed-Forward Neural Network.

5. Feed Forward Neural Network (Classifier Head)

1

Input

(1, 384) — combined pooled representation.



Performance Comparison

Model	Top-1 Accuracy	Top-5 Accuracy	Notes
Transformer Encoder (1.5M params)	55%	80%	4 layers, d_model=192, nhead=4, dim_feedforward=512, dropout=0.2
BiLSTM (2.8M params)	36%	68%	2 layers, hidden=256, bidirectional — ~2× more params but worse accuracy

Key insight: the Transformer (1.5M params) outperforms the BiLSTM (2.8M params) by roughly 19 percentage points in top-1 accuracy despite being nearly half the size. Self-attention's ability to model long-range temporal dependencies is critical for sign language recognition. Both models use the same 227-dim keypoint input features and 64-frame resampled sequences.

End-to-End Dimension Transformation Summary

Stage	Transformation	Shape Change
1-Frame	Read frame via OpenCV	(H, W, 3)
1-Color	BGR → RGB conversion	(H, W, 3)
1-MP	MediaPipe PoseLandmarker + HandLandmarker detection	33×3 + 2×(21×3)
1-Norm	Shoulder-center normalization + scale	75 × 3
1-Flat	Flatten + concat presence flags	(227,)
1-Stack	Repeat for T frames and stack	(T, 227)
2-Resamp	Linear interpolation to fix 64 frames	(64, 227)
3-Proj	Linear(227→192) + LayerNorm + Dropout	(1, 64, 192)
3-PE	Add sinusoidal positional encoding	(1, 64, 192)
3-Enc×4	4× TransformerEncoderLayer (MHA + FFN)	(1, 64, 192)
4-Mean	Mean pool over frames	(1, 192)
4-Attn	Attention pool (learned query)	(1, 192)
4-Concat	Concatenate [attn_pooled, mean_pooled]	(1, 384)
5-FFN1	Linear(384→192) + GELU + Dropout	(1, 192)
5-FFN2	Linear(192→100)	(1, 100)
5-Softmax	Softmax over 100 classes	(1, 100)
5-TopK	Argsort → top-3 classes → lookup label_map	3 strings + floats